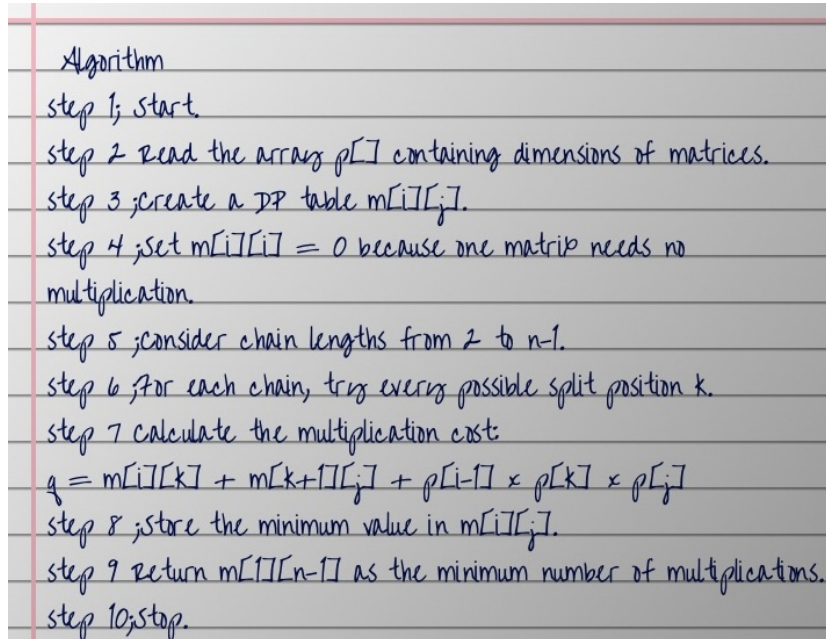


Practical 6

AIM: Implementation of chain matrix multiplication using dynamic programming.

1 ALGORITHM:



Algorithm
step 1; start.
step 2 Read the array $p[]$ containing dimensions of matrices.
step 3 ;create a DP table $m[i][j]$.
step 4 ;set $m[i][i] = 0$ because one matrix needs no multiplication.
step 5 ;consider chain lengths from 2 to $n-1$.
step 6 ;for each chain, try every possible split position k .
step 7 calculate the multiplication cost:
 $q = m[i][k] + m[k+1][j] + p[i-1] \times p[k] \times p[j]$
step 8 ;store the minimum value in $m[i][j]$.
step 9 return $m[1][n-1]$ as the minimum number of multiplications.
step 10; stop.

2 PROGRAM:

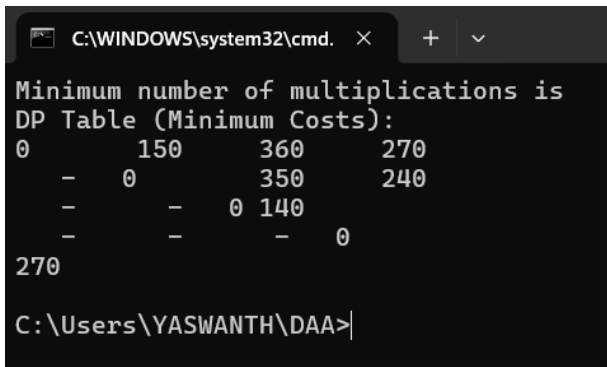
```
#include <iostream>
#include <climits>
using namespace std;
void printTable(int m[][100], int n) {
    cout << "\nDP Table (Minimum Costs):\n";
    for (int i = 1; i < n; i++) {
        for (int j = 1; j < n; j++) {
            if (i > j) cout << "    -    ";
            else cout << m[i][j] << "\t";
        }
        cout << endl;
    }
}
int matrixChainOrder(int p[], int n) {
    int m[100][100];
    for (int i = 1; i < n; i++)
        m[i][i] = 0;
    for (int L = 2; L < n; L++) {
        for (int i = 1; i < n - L + 1; i++) {
            int j = i + L - 1;
            m[i][j] = INT_MAX;
            for (int k = i; k < j; k++) {
                int q = m[i][k] + m[k+1][j] + p[i-1]*p[k]*p[j];
                if (q < m[i][j])
```

```

        m[i][j] = q;
    }
}
printTable(m, n);
return m[1][n-1];}
int main() {
    int arr[] = {3, 5, 10, 7, 2};
    int n = sizeof(arr)/sizeof(arr[0]);
    cout << "Minimum number of multiplications is "
        << matrixChainOrder(arr, n) << endl;
    return 0;}

```

3 OUTPUT:

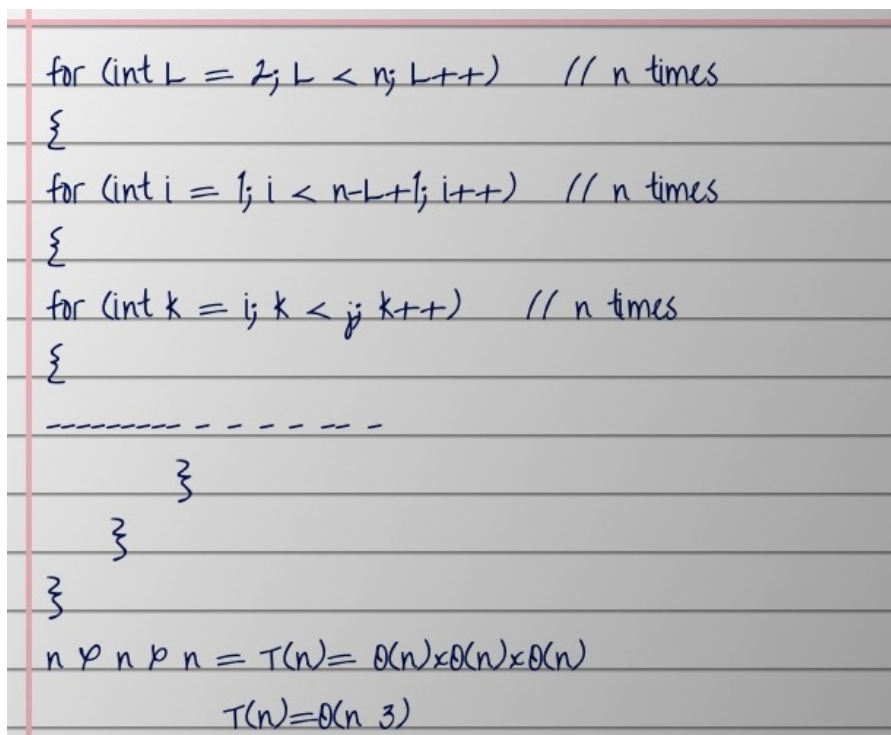


```

C:\WINDOWS\system32\cmd. x + v
Minimum number of multiplications is
DP Table (Minimum Costs):
0      150      360      270
-      0        350      240
-      -        0      140
-      -        -        0
270
C:\Users\YASWANTH\DAA>

```

4 TIME COMPLEXITY:



```

for (int L = 2; L <= n; L++) // n times
{
    for (int i = 1; i <= n-L+1; i++) // n times
    {
        for (int k = i; k <= i+L-1; k++) // n times
        {
            // ...
        }
    }
}
n * n * n = T(n) = O(n) * O(n) * O(n)
T(n) = O(n^3)

```

5 Conclusion:

Matrix Chain Multiplication using Dynamic Programming efficiently finds the minimum cost of multiplying a chain of matrices by solving subproblems optimally.